

HW 7 Cohort 1 Group 5

Saturday, February 28, 2026 10:15 PM

Shannon Carroll, Yingqi Fang, Xiaoyue Wang, Katherine Jin, Aarya Parashar, Julie

```
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
from scipy.optimize import minimize
import matplotlib.pyplot as plt
```

```
raw = pd.read_csv("C:/Users/secar/OneDrive/Documents/UCLA/Empirical Methods/HM7FFData.csv")
```

```
raw.head(15)
```

This file was created using the 202601 Bloomberg database.							
	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	Missing data are indicated by -99.99.	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	Mkt-RF	SMB	HML	RMW	CMA	RF
6	199007	1.99	2.63	0.38	-0.56	0.58	0.68
7	199008	-11.15	-0.62	0.27	1.09	0.01	0.66
8	199009	-14.55	3.31	1.27	-0.25	0.79	0.6
9	199010	15.04	-9.3	-5.6	2.08	2.37	0.68
10	199011	-8.02	1.5	1.91	2.19	-1.69	0.57
11	199012	0.44	-2.29	-1.83	1.73	0.96	0.6
12	199101	1.4	-4.57	0.19	1.69	-0.67	0.52
13	199102	10.01	1.85	0.71	-2.26	0.03	0.48
14	199103	-4.76	2.23	-0.2	1.29	-0.99	0.44

```
import pandas as pd
```

```
path = "C:/Users/secar/OneDrive/Documents/UCLA/Empirical Methods/HM7FFData.csv"
```

```
df = pd.read_csv(path, skiprows=6)
```

```
df.rename(columns={df.columns[0]: "Date"}, inplace=True)
```

```
df.columns = df.columns.str.strip()
```

```
print(df.columns)
```

```
print(df.head())
```

```
Index(['Date', 'Mkt-RF', 'SMB', 'HML', 'RMW', 'CMA', 'RF'], dtype='object')
```

```

Date Mkt-RF SMB HML RMW CMA RF
0 199007 1.99 2.63 0.38 -0.56 0.58 0.68
1 199008 -11.15 -0.62 0.27 1.09 0.01 0.66
2 199009 -14.55 3.31 1.27 -0.25 0.79 0.6
3 199010 15.04 -9.3 -5.6 2.08 2.37 0.68
4 199011 -8.02 1.5 1.91 2.19 -1.69 0.57
```

```

df["Date"] = df["Date"].astype(str).str.strip()
df = df[df["Date"].str.fullmatch(r"\d{6}").copy()]
df["Date"] = pd.to_datetime(df["Date"], format="%Y%m")
df = df.set_index("Date")

df = df.apply(pd.to_numeric, errors="coerce") / 100.0

df.columns = df.columns.str.strip()

cma = df["CMA"].dropna()
print(cma.index.min(), cma.index.max())
print(cma.head())

1990-07-01 00:00:00 2026-01-01 00:00:00
Date
1990-07-01    0.0058
1990-08-01    0.0001
1990-09-01    0.0079
1990-10-01    0.0237
1990-11-01   -0.0169
Name: CMA, dtype: float64

from statsmodels.tsa.arima.model import ARIMA

arma = ARIMA(cma, order=(1,0,1), trend="c").fit()
print(arma.summary())

C:\Users\seca\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: Va
ency MS will be used.
  self._init_dates(dates, freq)
C:\Users\seca\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: Va
ency MS will be used.
  self._init_dates(dates, freq)
C:\Users\seca\anaconda3\lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: Va
ency MS will be used.
  self._init_dates(dates, freq)

SARIMAX Results
=====
Dep. Variable:          CMA      No. Observations:          427
Model:                ARIMA(1, 0, 1)      Log Likelihood        1149.383
Date:                Sat, 28 Feb 2026      AIC                  -2290.766
Time:                21:27:02             BIC                  -2274.539
Sample:              07-01-1990          HQIC                  -2284.357
               - 01-01-2026

Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
const          0.0017      0.001      1.187      0.235      -0.001     0.004
ar.L1          0.6060      0.068      8.872      0.000      0.472     0.740
ma.L1         -0.3145      0.090     -3.478      0.001     -0.492    -0.137
sigma2         0.0003      1.19e-05    22.503      0.000      0.000     0.000
=====
Ljung-Box (L1) (Q):                0.00      Jarque-Bera (JB):                145.57
Prob(Q):                          0.99      Prob(JB):                      0.00
Heteroskedasticity (H):            0.47      Skew:                          0.19
Prob(H) (two-sided):              0.00      Kurtosis:                       5.83
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step)

phi = arma.params["ar.L1"]
print("AR(1) persistence (phi):", phi)

```

```

AR(1) persistence (phi): 0.6059812067583126

half_life = np.log(0.5) / np.log(abs(phi))
print("Half-life (months):", half_life)
Half-life (months): 1.3837860956538819

eps = arma.resid.dropna()

from statsmodels.tsa.api import arch_model

-----
ImportError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 from statsmodels.tsa.api import arch_model

ImportError: cannot import name 'arch_model' from 'statsmodels.tsa.api' (C

from statsmodels.tsa.arima.model import ARIMA

eps = np.asarray(eps)
T = len(eps)

from scipy.optimize import minimize

def garch11_negloglik(params, eps):
    # unconstrained
    a0, a1, a2 = params

    # enforce omega>0
    omega = np.exp(a0)

    # map (a1,a2) -> alpha,beta >=0 and alpha+beta<1
    u = np.exp(a1)
    v = np.exp(a2)
    denom = 1 + u + v
    alpha = u / denom
    beta = v / denom

    T = len(eps)
    sigma2 = np.empty(T)

    # initialize at unconditional variance
    sigma2[0] = omega / (1 - alpha - beta)

    for t in range(1, T):
        sigma2[t] = omega + alpha * eps[t-1]**2 + beta * sigma2[t-1]
        if sigma2[t] <= 0 or not np.isfinite(sigma2[t]):
            return 1e12

    # Log-Likelihood
    ll = -0.5 * np.sum(np.log(2*np.pi) + np.log(sigma2) + (eps**2)/sigma2)
    return -ll # minimize negative log-likelihood

init = np.array([np.log(np.var(eps)*0.05), np.log(0.05), np.log(0.90)]) # starting guess
res_g = minimize(garch11_negloglik, init, args=(eps,), method="Nelder-Mead")

a0, a1, a2 = res_g.x
omega = np.exp(a0)
u, v = np.exp(a1), np.exp(a2)
denom = 1 + u + v
alpha = u/denom
beta = v/denom

print("GARCH(1,1) estimates:")
print("omega =", omega)
print("alpha =", alpha)
print("beta =", beta)
print("alpha+beta =", alpha+beta)
print("Converged:", res_g.success, "|", res_g.message)

GARCH(1,1) estimates:
omega = 2.1033173235351513e-05
alpha = 0.33377105518530037
beta = 0.6114707582260792
alpha+beta = 0.9452418134113796
Converged: True | Optimization terminated successfully.

sigma2_g = np.empty(T)
sigma2_g[0] = omega/(1-alpha-beta)
for t in range(1, T):
    sigma2_g[t] = omega + alpha*eps[t]**2 + beta*sigma2_g[t-1]

def arch12_negloglik(params, eps, p=12):
    # params: [a0, a1..ap] unconstrained
    a0 = params[0]
    a = params[1:]

    omega = np.exp(a0)

    u = np.exp(a)           # positive
    denom = 1 + np.sum(u)
    alphas = u / denom      # sum(alphas) < 1 automatically

    T = len(eps)
    sigma2 = np.empty(T)
    eps2 = eps**2

    # initialize first p values with unconditional variance
    sigma2[:p] = omega / (1 - np.sum(alphas))

    for t in range(p, T):
        sigma2[t] = omega + np.dot(alphas, eps2[t-p:t][:-1])
        if sigma2[t] <= 0 or not np.isfinite(sigma2[t]):
            return 1e12

    ll = -0.5 * np.sum(np.log(2*np.pi) + np.log(sigma2[p:]) + eps2[p:]/sigma2[p:])
    return -ll

```

```

p = 12
init = np.concatenate([(np.log(np.var(eps)*0.05)), np.log(np.full(p, 0.02))])
res_a = minimize(arch12_negloglik, init, args=(eps,p), method="Nelder-Mead")

a0 = res_a.x[0]
a = res_a.x[1:]
omega_a = np.exp(a0)
u = np.exp(a)
alphas_a = u / (1 + np.sum(u))

print("ARCH(12) estimates:")
print("omega =", omega_a)
print("sum(alpha_1) =", np.sum(alphas_a))
print("alphas =", alphas_a)
print("Converged:", res_a.success, "|", res_a.message)

ARCH(12) estimates:
omega = 4.8471042008299675e-05
sum(alpha_1) = 0.9324995504454273
alphas = [3.66852898e-01 2.50654724e-01 1.67394282e-01 1.38403238e-02
 3.12245330e-02 3.72388299e-02 1.07776946e-04 4.75531993e-04
 1.2480802e-06 2.62701518e-07 6.52456985e-02 5.34409792e-05]
Converged: False | Maximum number of function evaluations has been exceeded.

sigma2_a = np.empty(T)
eps2 = eps**2
sigma2_a[:p] = omega_a/(1-np.sum(alphas_a))
for t in range(p, T):
    sigma2_a[t] = omega_a + np.dot(alphas_a, eps2[t-p:t][::-1])

idx = arma.resid.dropna().index

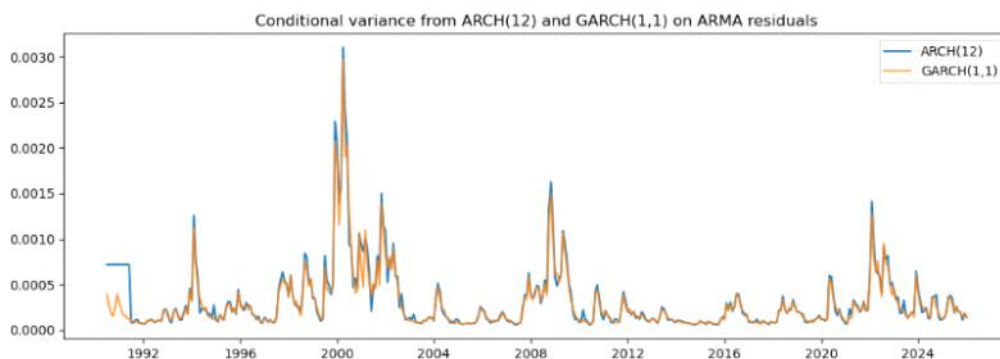
import matplotlib.pyplot as plt

plt.figure(figsize=(11,4))
plt.plot(idx, sigma2_a, label="ARCH(12)")
plt.plot(idx, sigma2_g, label="GARCH(1,1)", alpha=0.8)
plt.legend()
plt.title("Conditional variance from ARCH(12) and GARCH(1,1) on ARMA residuals")
plt.tight_layout()
plt.show()

```

- For the GARCH(1,1) model, $\alpha + \beta < 1$, indicating covariance stationarity. That also guarantees unconditional variance. The ARCH(12) process also satisfies the stationarity condition (sum of ARCH coefficients < 1).

Variance plot interpretation: Both models capture volatility clustering, with large spikes during crisis periods (dot-com bubble, global financial crisis, COVID-19). The GARCH(1,1) model produces smoother and more persistent variance dynamics compared to ARCH(12). ARCH is slightly more reactive. Memory is limited to 12 lags and variance decays faster after shocks.



```

from statsmodels.tsa.stattools import acf

# keep idx from arma.resid.dropna().index
idx = arma.resid.dropna().index
eps = np.asarray(arma.resid.dropna())

# conditional sigmas
sigma_a = np.sqrt(sigma2_a)
sigma_g = np.sqrt(sigma2_g)

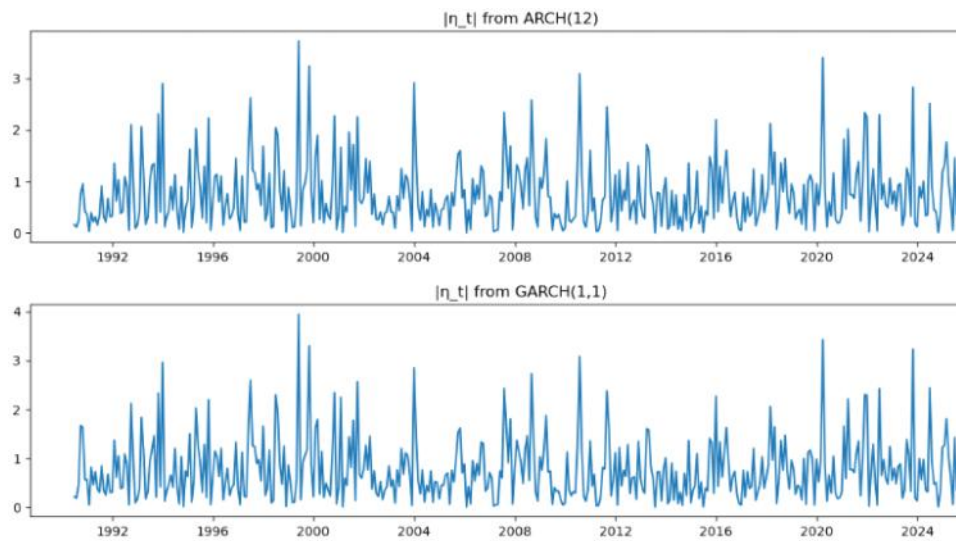
# normalized residuals
eta_a = eps / sigma_a
eta_g = eps / sigma_g

abs_eta_a = np.abs(eta_a)
abs_eta_g = np.abs(eta_g)

plt.figure(figsize=(11,3))
plt.plot(idx, abs_eta_a)
plt.title("|n_t| from ARCH(12)")
plt.tight_layout()
plt.show()

plt.figure(figsize=(11,3))
plt.plot(idx, abs_eta_g)
plt.title("|n_t| from GARCH(1,1)")
plt.tight_layout()
plt.show()

```



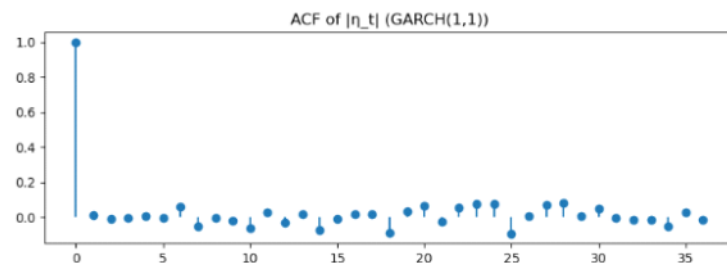
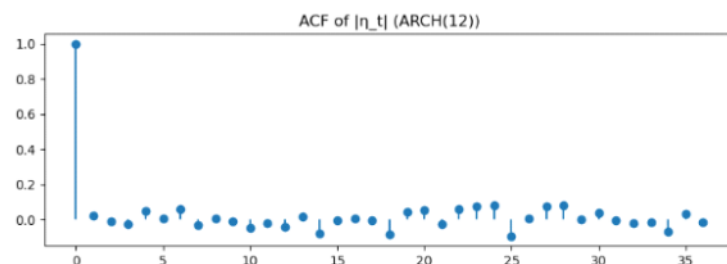
max_lag = 36 # monthly data: 36 lags = 3 years

```
acf_a = acf(abs_eta_a, nlags=max_lag, fft=True)
acf_g = acf(abs_eta_g, nlags=max_lag, fft=True)
```

```
plt.figure(figsize=(8,3))
plt.stem(range(len(acf_a)), acf_a, basefmt=" ")
plt.title("ACF of |η_t| (ARCH(12))")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8,3))
plt.stem(range(len(acf_g)), acf_g, basefmt=" ")
plt.title("ACF of |η_t| (GARCH(1,1))")
plt.tight_layout()
plt.show()
```

3. The autocorrelation of $|\eta_t|$ is substantially reduced relative to ε_t^2 , suggesting that both ARCH(12) and GARCH(1,1) adequately account for volatility clustering. Spikes do not appear to be too clustered.



ARCH(12) and GARCH(1, 1) has removed almost all volatility clustering. Lag 0 = 1 by definition, and remaining are close to 0. There is no visible persistence left in the magnitude of shocks.

```
path_daily = "C:/Users/sear/OneDrive/Documents/UCLA/Empirical Methods/Developed_ex_US_5_Factors_Daily.csv"
d = pd.read_csv(path_daily, skiprows=6)
d.rename(columns={d.columns[0]: "Date"}, inplace=True)
d.columns = d.columns.str.strip()
```

```
# keep only daily YYYYMMDD rows
d["Date"] = d["Date"].astype(str).str.strip()
d = d[d["Date"].str.fullmatch(r"\d{8}").copy()]
```

```
d["Date"] = pd.to_datetime(d["Date"], format="%Y%m%d")
d = d.set_index("Date")
```

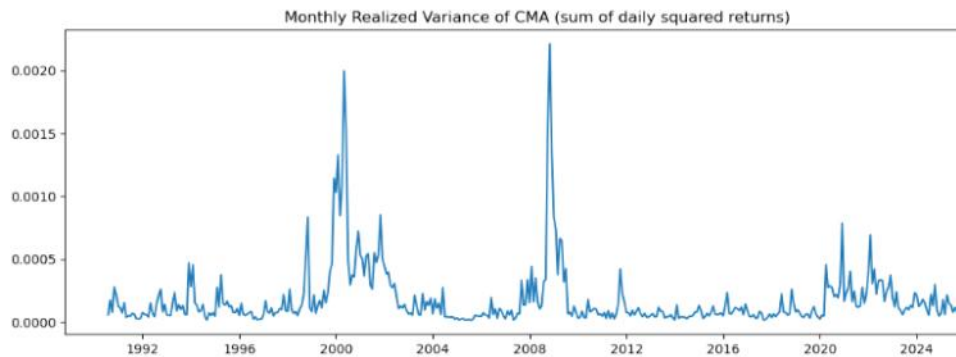
```
# percent -> decimal
d = d.apply(pd.to_numeric, errors="coerce") / 100.0
d.columns = d.columns.str.strip()
```

```
cma_d = d["CMA"].dropna()
```

```
# realized variance per month: sum of squared daily returns in that month
RV = (cma_d**2).resample("M").sum()
```



```
plt.figure(figsize=(11,4))
plt.plot(RV.index, RV.values)
plt.title("Monthly Realized Variance of CMA (sum of daily squared returns)")
plt.tight_layout()
plt.show()
```



```
print(len(dfq))
print(dfq.head())

0
Empty DataFrame
Columns: [RV, eps, sigma2_garch, eps2]
Index: []
```

```
print("RV:", RV.index.min(), RV.index.max(), RV.index[:5])
print("eps:", eps_series.index.min(), eps_series.index.max(), eps_series.index[:5])
print("sigma2:", sigma2_series.index.min(), sigma2_series.index.max(), sigma2_series.index[:5])
print(type(RV.index), RV.index.dtype)
print(type(eps_series.index), eps_series.index.dtype)
```

```
RV: 1990-07-31 00:00:00 2026-01-31 00:00:00 DatetimeIndex(['1990-07-31', '1990-08-31', '1990-09-30', '1990-10-31',
1990-11-30'],
dtype='datetime64[ns]', name='Date', freq='M')
eps: 1990-07-01 00:00:00 2026-01-01 00:00:00 DatetimeIndex(['1990-07-01', '1990-08-01', '1990-09-01', '1990-10-01',
1990-11-01'],
dtype='datetime64[ns]', name='Date', freq=None)
sigma2: 1990-07-01 00:00:00 2026-01-01 00:00:00 DatetimeIndex(['1990-07-01', '1990-08-01', '1990-09-01', '1990-10-01',
1990-11-01'],
dtype='datetime64[ns]', name='Date', freq=None)
<class 'pandas.core.indexes.datetimes.DatetimeIndex'> datetime64[ns]
<class 'pandas.core.indexes.datetimes.DatetimeIndex'> datetime64[ns]
```

```
RV_fixed = RV.copy()
RV_fixed.index = pd.to_datetime(RV_fixed.index).to_period("M").to_timestamp("M")

eps_fixed = eps_series.copy()
eps_fixed.index = pd.to_datetime(eps_fixed.index).to_period("M").to_timestamp("M")

sigma_fixed = sigma2_series.copy()
sigma_fixed.index = pd.to_datetime(sigma_fixed.index).to_period("M").to_timestamp("M")
```

```
dfq = pd.concat(
    [RV_fixed.rename("RV"), eps_fixed.rename("eps"), sigma_fixed.rename("sigma2_garch")],
    axis=1
).dropna()
```

```
print("Rows after alignment:", len(dfq))
print(dfq.head())
```

```
Rows after alignment: 427
RV      eps  sigma2_garch
Date
1990-07-31  0.000056  0.004136  0.000384
1990-08-31  0.000174 -0.002923  0.000262
1990-09-30  0.000076  0.006275  0.000184
1990-10-31  0.000278  0.020229  0.000147
1990-11-30  0.000215 -0.025555  0.000247
```

```
from statsmodels.tsa.arima.model import ARIMA
import pandas as pd
import matplotlib.pyplot as plt

rv = dfq[["RV"]]

arma_rv = ARIMA(rv, order=(1,0,1), trend="c").fit()
print(arma_rv.summary())
```

```
SARIMAX Results
=====
Dep. Variable:          RV      No. Observations:          427
Model:                ARIMA(1, 0, 1)      Log Likelihood      3137.562
Date:                  Sat, 28 Feb 2026      AIC                -6267.124
Time:                  21:53:28      BIC                -6250.896
Sample:                07-31-1990      HQIC               -6260.714
                   - 01-31-2026

Covariance Type:      opg
=====
              coef  std err          z      P>|z|      [0.025      0.975]
-----
const          0.0002   5.1e-05     3.473     0.001   7.71e-05     0.000
ar.L1          0.7795   0.011    74.093     0.000     0.759     0.800
ma.L1         -0.0009   0.007    -0.132     0.895    -0.014     0.012
sigma2        2.417e-08  5.99e-10   40.339     0.000   2.3e-08   2.53e-08
=====
Ljung-Box (L1) (Q):                0.00  Jarque-Bera (JB):            8543.67
Prob(Q):                          0.98  Prob(JB):                  0.00
Heteroskedasticity (H):             0.32  Skew:                      2.81
Prob(H) (two-sided):              0.00  Kurtosis:                   24.18
=====
```

```
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
[2] Covariance matrix is singular or near-singular, with condition number 1.15e+18. Standard errors may be unstable.
```

```
v = arma_rv.fittedvalues.rename("v")
```

Since RV aggregates many daily squared returns within a month, it is a less noisy proxy for latent variance than ϵ^2 , which is only one squared innovation. Therefore, RV is a much better estimator of true variance.

Warnings:
 [1] Covariance matrix calculated using the outer product of gradients (complex-step).
 [2] Covariance matrix is singular or near-singular, with condition number 1.15e+18. Standard errors may be unstable.

```
v = arma_rv.fittedvalues.rename("v")

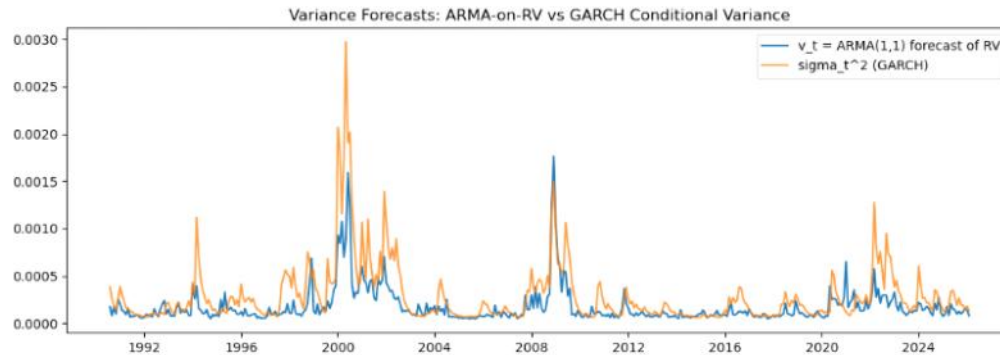
# compare to RV and GARCH sigma^2
df6 = pd.concat([rv.rename("RV"), v, dfq["sigma2_garch"].rename("sigma2_garch")], axis=1).dropna()

corr_v_RV = df6["v"].corr(df6["RV"])
corr_sigma2_RV = df6["sigma2_garch"].corr(df6["RV"])

print("Corr(v_t, RV_t):", corr_v_RV)
print("Corr(sigma^2_GARCH_t, RV_t):", corr_sigma2_RV)

Corr(v_t, RV_t): 0.7787888130993934
Corr(sigma^2_GARCH_t, RV_t): 0.7236358038648285

plt.figure(figsize=(11,4))
plt.plot(df6.index, df6["v"], label="v_t = ARMA(1,1) forecast of RV")
plt.plot(df6.index, df6["sigma2_garch"], label="sigma_t^2 (GARCH)", alpha=0.8)
plt.title("Variance Forecasts: ARMA-on-RV vs GARCH Conditional Variance")
plt.legend()
plt.tight_layout()
plt.show()
```



```
rv = dfq["RV"]
arma_rv = ARIMA(rv, order=(1,0,1), trend="c").fit()
print(arma_rv.summary())

# v_t = E_t-1[RV_t] one-step-ahead forecast
v = arma_rv.get_prediction(start=rv.index[1], dynamic=False).predicted_mean
v = v.rename("v")

# align v with RV and sigma2
df6 = pd.concat([rv.rename("RV"), v, dfq["sigma2_garch"].rename("sigma2_garch")], axis=1).dropna()

corr_v_RV = df6["v"].corr(df6["RV"])
corr_sigma2_RV = df6["sigma2_garch"].corr(df6["RV"])

print("Corr(v_t, RV_t):", corr_v_RV)
print("Corr(sigma^2_GARCH_t, RV_t):", corr_sigma2_RV)

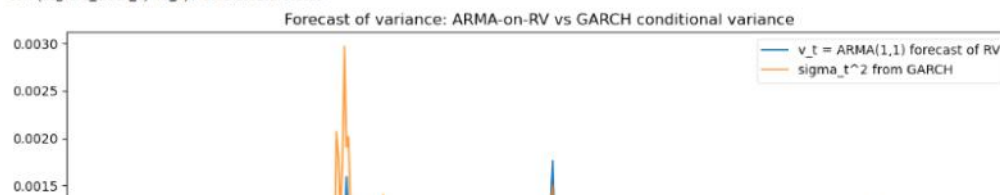
plt.figure(figsize=(11,4))
plt.plot(df6.index, df6["v"], label="v_t = ARMA(1,1) forecast of RV")
plt.plot(df6.index, df6["sigma2_garch"], label="sigma_t^2 from GARCH", alpha=0.8)
plt.title("Forecast of variance: ARMA-on-RV vs GARCH conditional variance")
plt.legend()
plt.tight_layout()
plt.show()
```

SARIMAX Results

```
=====
Dep. Variable:          RV      No. Observations:      427
Model:                ARIMA(1, 0, 1)      Log Likelihood:    3137.562
Date:                Sat, 28 Feb 2026      AIC:              -6267.124
Time:                21:55:45              BIC:              -6250.896
Sample:              07-31-1990           HQIC:             -6260.714
Sample:              - 01-31-2026

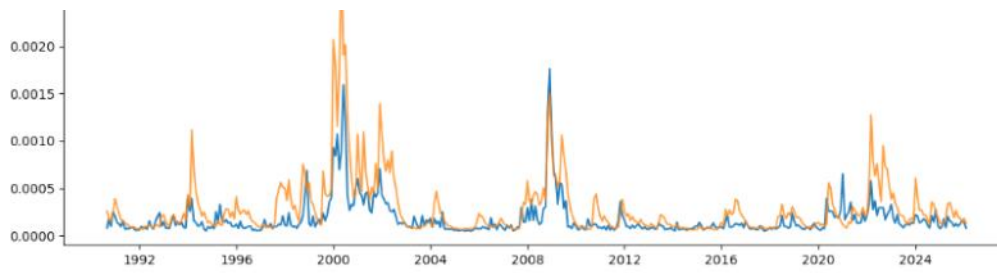
Covariance Type:      opg
=====
              coef  std err      z      P>|z|      [0.025   0.975]
-----
const          0.0002    5.1e-05    3.473    0.001    7.71e-05    0.000
ar.L1          0.7795    0.011   74.093    0.000    0.759    0.800
ma.L1         -0.0009    0.007   -0.132    0.895   -0.014    0.012
sigma2        2.417e-08  5.99e-10  40.339    0.000    2.3e-08    2.53e-08
=====
Ljung-Box (L1) (Q):                0.00  Jarque-Bera (JB):        8543.67
Prob(Q):                          0.98  Prob(JB):                0.00
Heteroskedasticity (H):            0.32  Skew:                  2.81
Prob(H) (two-sided):              0.00  Kurtosis:              24.18
=====
```

Warnings:
 [1] Covariance matrix calculated using the outer product of gradients (complex-step).
 [2] Covariance matrix is singular or near-singular, with condition number 1.15e+18. Standard errors may be unstable.
 Corr(v_t, RV_t): 0.7789981281411692
 Corr(sigma^2_GARCH_t, RV_t): 0.7242592037956509



6) The correlation between v_t and RV_t is higher than the correlation between σ_t^2 (GARCH) and RV_t .

This indicates that the ARMA(1,1) model applied to realized variance provides a better



σ_t^2 (GARCH) and RV_t .

This indicates that the ARMA(1,1) model applied to realized variance provides a better forecast of future variance than the GARCH model based on squared residuals.